

وثيقة المراجعة والتلخيص الأكاديمي

Source: publisher companion PDFs (3rd ed., New Age International, 2007).

Doctor covered ~3 chapters. These are the book's own review questions — high exam value.

Ch.1 Introduction

Ch.1 Introduction p.61

Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 61 Exercises 1.1 Why is primary goal of software development now shifting from producing good quality software to good quality maintainable software? 1.2 List the reasons for the "software crisis"? Why are CASE tools not normally able to control it? 1.3 "The software crisis is aggravated by the progress in hardware technology?" Explain with examples. 1.4 What is software crisis? Was Y2K a software crisis? 1.5 What is the significance of software crisis in reference to software engineering discipline. 1.6 How are software myths affecting software process? Explain with the help of examples. 1.7 State the difference between program and software. Why have documents and documentation become very important. 1.8 What is software engineering? Is it an art, craft or a science? Discuss.

Ch.1 Introduction p.62

Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 62 Exercises 1.9 What is aim of software engineering? What does the discipline of software engineering discuss? 1.10 Define the term "Software engineering". Explain the major differences between software engineering and other traditional engineering disciplines. 1.11 What is software process? Why is it difficult to improve it? 1.12 Describe the characteristics of software contrasting it with the characteristics of hardware. 1.13 Write down the major characteristics of a software. Illustrate with a diagram that the software does not wear out. 1.14 What are the components of a software? Discuss how a software differs from a program. 1.15 Discuss major areas of the applications of the software. 1.16 Is software a product or process? Justify your answer with example

Ch.1 Introduction p.63

Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 63 Exercises 1.17 Differentiate between the following (i) Deliverables and milestones (ii) Product and process (iii) Measures, metrics and measurement 1.18 What is software metric? How is it different from software measurement 1.19 Discuss software process and product metrics with the help of examples. 1.20 What is productivity? How is it related to effort. What is the unit of effort. 1.21 Differentiate between module and software component. 1.22 Distinguish between generic and customized software products. Which one has larger share of market and why? 1.23 Is software a product or process? Justify your answer with example

Ch.1 Introduction p.64

Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 64 1.23 Describe the role of management in software development with the help of examples. 1.24 What are various factors of management dependency in software development. Discuss each factor in detail. 1.25 What is more important: Product or process? Justify your answer. Exercises

Ch.2 SDLC Models

Ch.2 SDLC Models p.46

46 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 2.1 What do you understand by the term Software Development Life Cycle (SDLC)? Why is it important to adhere to a life cycle model while developing a large software product? 2.2 What is software life cycle? Discuss the generic waterfall model. 2.3 List the advantages of using waterfall model in stead of adhoc build and fix model. 2.4 Discuss the prototyping model. What is the effect of designing a prototype on the overall cost of the project? 2.5 What are the advantages of developing the prototype of a system? 2.6 Describe the type of situations where iterative enhancement model might lead to difficulties. 2.7 Compare iterative enhancement model and evolutionary process model.

Ch.2 SDLC Models p.47

47 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 2.8 Sketch a neat diagram of spiral model of software life cycle. 2.10 As we move outward along with process flow path of the spiral model, what can we say about software that is being developed or maintained. 2.11 How does “project risk” factor effect the spiral model of software development. 2.12 List the advantages and disadvantages of involving a software engineer throughout the software development planning process. 2.13 Explain the spiral model of software development. What are the limitations of such a model? 2.14 Describe the rapid application development (RAD) model. Discuss each phase in detail. 2.15 What are the characteristics to be considered for the selection of the life cycle model? 2.9 Compare the waterfall model and the spiral model of software development.

Ch.2 SDLC Models p.48

48 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 2.16 What is the role of user participation in the selection of a life cycle model?. 2.18 Write short note on “status of development team” for the selection of a life cycle model?. 2.19 Discuss the selection process parameters for a life cycle model. 2.20 What is unified process? Explain various phases along with the outcome of each phase. 2.21 Describe the unified process work products after each phase of unified process. 2.22 What are the advantages of iterative approach over sequential approach? Why is unified process called as iterative or incremental? 2.17 Why do we feel that characteristics of requirements play a very significant role in the selection of a life cycle model?

Ch.3 Software Requirements / SRS

Ch.3 Software Requirements / SRS p.134

134 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.1 Discuss the significance and use of requirement engineering. What are the problems in the formulation of requirements? 3.2 Requirements analysis is unquestionably the most communication intensive step in the software engineering process. Why does the communication path frequently break down ? 3.3 What are crucial process steps of requirement engineering ? Discuss with the help of a diagram. 3.4 Discuss the present state of practices in requirement engineering. Suggest few steps to improve the present state of practice. 3.5 Explain the importance of requirements. How many types of requirements are possible and why ? 3.7 What do you understand with the term "requirements elicitation" ? Discuss any two techniques in detail. 3.8 List out requirements elicitation techniques. Which one is most popular and why ? 3.6 Describe the various steps of requirements engineering. Is it essential to follow these steps ?

Ch.3 Software Requirements / SRS p.135

135 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.10 Discuss quality function deployment technique of requirements elicitation. Why an importance or value factor is associated with every requirement ? 3.11. Explain the use case approach of requirements elicitation. What are use-case guidelines ? 3.12. What are components of a use case diagram. Explain their usage with the help of an example. 3.13. Consider the problem of library management system and design the following: (i) Problem statement (ii) Use case diagram (iii) Use cases. 3.9 Describe facilitated application specification technique (FAST) and compare this with brainstorming sessions.

Ch.3 Software Requirements / SRS p.136

136 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.14. Consider the problem of railway reservation system and design the following: (i) Problem statement (ii) Use case diagram (iii) Use cases. 3.15. Explain why a many to many relationship is to be modeled as an associative entity ? 3.16. What are the linkages between data flow and E-R diagrams ? 3.17. What is the degree of a relationship ? Give an example of each of the relationship degree. 3.18. Explain the relationship between minimum cardinality and optional and mandatory participation. 3.19. An airline reservation is an association between a passenger, a flight, and a seat. Select a few pertinent attributes for each of these entity types and represent a reservation in an E-R diagram.

Ch.3 Software Requirements / SRS p.137

137 Software Engineering (3 rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.20. A department of computer science has usual resources and usual users for these resources. A software is to be developed so that resources are assigned without conflict. Draw a DFD specifying the above system. 3.21. Draw a DFD for result preparation automation system of B. Tech. courses (or MCA program) of any university. Clearly describe the working of the system. Also mention all assumptions made by you. 3.22. Write short notes on (i) Data flow diagram (ii) Data dictionary. 3.23. Draw a DFD for borrowing a book in a library which is explained below: "A borrower can borrow a book if it is available else he/she can reserve for the book if he/she so wishes. He/she can borrow a maximum of three books". 3.24. Draw the E-R diagram for a

hotel reception desk management. Explain why, for large software systems development, is it recommended that prototypes should be “throw-away” prototype?

Ch.3 Software Requirements / SRS p.138

138 Software Engineering (3rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.26. Discuss the significance of using prototyping for reusable components and explain the problems, which may arise in this situation. 3.27. Suppose a user is satisfied with the performance of a prototype. If he/she is interested to buy this for actual work, what should be the response of a developer? 3.28. Comment on the statement: “The term throw-away prototype is inappropriate in that these prototypes expand and enhance the knowledge base that is retained and incorporated in the final prototype; therefore they are not disposed of or thrown away at all.” 3.29. Which of the following statements are ambiguous? Explain why. (a) The system shall exhibit good response time. (b) The system shall be menu driven. (c) There shall exist twenty-five buttons on the control panel, numbered PF1 to PF25. (d) The software size shall not exceed 128K of RAM.

Ch.3 Software Requirements / SRS p.139

139 Software Engineering (3rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.30. Are there other characteristics of an SRS (besides listed in section 3.4.2) that are desirable? List a few and describe why? 3.31. What is software requirements specification (SRS)? List out the advantages of SRS standards. Why is SRS known as the black box specification of a system? 3.32. State the model of a data dictionary and its contents. What are its advantages? 3.33. List five desirable characteristics of a good SRS document. Discuss the relative advantages of formal requirement specifications. List the important issues, which an SRS must address. 3.34. Construct an example of an inconsistent (incomplete) SRS. 3.35. Discuss the organization of a SRS. List out some important issues of this organization.

Ch.3 Software Requirements / SRS p.140

140 Software Engineering (3rd ed.), By K.K Aggarwal & Yogesh Singh, Copyright © New Age International Publishers, 2007 Exercises 3.36. Discuss the difference between the following: (a) Functional & nonfunctional requirements (b) User & system requirements

Retrieval set

Cover the answer, produce your own, then compare. Each item points at a question in Aggarwal & Singh, *Software Engineering* (3rd ed., New Age International, 2007); the page number is that of the exercise itself. The answers below are concise concept statements written from standard SE principles — not verbatim textbook quotations. [Foundational Knowledge / Standard Concept]

1 The primary goal of development is shifting from "good quality software" to "good quality maintainable software". Why? (Ch.1, p.61, Q1.1)

but the system keeps ,release. Hardware is cheap and replaceable Because a software's value now outlives its first A program that is excellent .**not construction ,evolution** so the cost that dominates is ,being modified for years
 a liability the moment the requirements change but undebuggable becomes
 صعوبة التعديل، والجودة بدون قابلية للصيانة تنطفئ مع أول طلب تعديل.
 التعليق: كل تغيير مستقبلي يدفع ثمن

2 Why are CASE tools normally unable to control the software crisis? (Ch.1, p.61, Q1.2)

CASE automates the **mechanics** of building (diagramming, code generation, version control) but cannot repair a decisions the — broken **process or requirements**. The crisis originates in estimation, communication and scope
 tool does not own .التعليق: الأداة تنظم العمل، ما تصلح فشل القرار.
 broken process or requirements

3 Was Y2K a software crisis? Justify. (Ch.1, p.61, Q1.4)

left unfixed ,a defect known for decades :the crisis rather than the crisis itself **symptom that exposed** Y2K was the
 It is the ."not urgent" under-prioritised and rewrites were deferred as because maintenance was perpetually
 huge remediation cost for code that still had to be — at national scale maintainability failure of Q1.1 arriving
 discarded or rewritten .التعليق: الأزمة ليست الخطأ، بل استحالة معالجته بأثر رخيص.

4 Illustrate that software does not wear out, and state the consequence. (Ch.1, p.62, Q1.12–1.13)

it fails only because of — Software has no physical wear mechanism, so its failure curve does not rise from ageing
 The .(regulation ,inputs ,OS ,hardware) around it **environment changed** or because the **latent defects surfacing**
 "old" and ,**preventive maintenance** degradation must be fought with :consequence is inverted from hardware
 software is not automatically untrustworthy .التعليق: العتاد يهرم، والبرمجيات تتقادم متطلباتها.

5 Is software a product, a process, or both? Which matters more? (Ch.1, p.62–64, Q1.16 / Q1.23 / Q1.25)

It is **both**: the *product* is the deliverable (program + data + documentation), the *process* is the ordered set of activities that produced it. **Process matters more**, because a sound process tolerates an imperfect team while an not an independent ,the product is an output of the process — unsound process fails even with a strong one
 achievement .التعليق: النتيجة قابلة للتحسين فقط عبر مصدرها، والبرنامج بلا المستندات نصف منتج.

6 Why must a team developing a large product adhere to a life cycle model? (Ch.2, p.46, Q2.1)

the entry/exit criteria ,the order of activities cycle model fixes Because scale destroys ad-hoc coordination. A life their origin so progress is measurable and defects are found near ,artefacts exchanged of each phase and the at large scale the disorder shows up as ;build-and-fix can hide the disorder rather than at delivery. For small work rework. التعليل: النموذج هو عقد التنسيق بين الفريق والمراجع.

7 List the advantages of waterfall over the ad-hoc build-and-fix model. (Ch.2, p.46, Q2.3)

makes ,forces requirements to be settled before coding ,explicit milestones and deliverables Waterfall gives on. Build-and-fix offers no and produces the documentation later phases depend ,review and estimation possible which is precisely why it collapses on non-trivial — means "done" anyone can say what checkpoint at which systems. التعليل: بدون مراحل محدّدة لا يمكن قياس التقدم ولا اكتشاف الخطأ مبكراً.

8 What is the effect of designing a prototype on the overall project cost? (Ch.2, p.46, Q2.4)

It raises cost in the short term and reduces it overall: the throw-away prototype is extra effort, but it converts an unknown (misunderstood requirements, unproven feasibility) into a known one before full-scale construction, when changes are still cheap. The cost of discovering a requirements error in construction or operation is an order of magnitude higher. التعليل: اكتشاف الخطأ في الرسم قبل ما تكتشفه بالبناء.

9 Compare the waterfall and spiral models, and explain what the outward spiral path implies about project risk. (Ch.2, p.47, Q2.9 / Q2.10 / Q2.11)

with each phase feeding the next. Spiral is ,test → build → specify → plan :linear and single-pass Waterfall is before committing to project risk evaluates every loop adds an increment and explicitly :risk-driven and cyclic each cycle retires — risk falls residual the product is being extended while the more work. Moving outward means instead of running once toward "complete" to "feasible" so the software matures from ,the riskiest assumption first an uncertain end. التعليل: الشلال يفترض أن المتطلبات صحيحة، واللؤلؤ يفترض العكس ويختبره.

10 Requirements analysis is the most communication-intensive step in software engineering — why does the communication path frequently break down? (Ch.3, p.134, Q3.2)

Because the two ends speak different languages and hold different mental models: users describe problems in domain terms (vague, exception-laden, context assumed), analysts must convert them into unambiguous, nobody volunteers what they consider — testable specifications. The path breaks at the unstated assumptions undetected so gaps and contradictions survive into analysis ,obvious كل طرف أن الطرف الآخر فاهم.

11 Which of these statements are ambiguous, and why? (Ch.3, p.138, Q3.29)

"menu driven" (b) .differs by context "good" and ,no measurable bound :ambiguous — "good response time" (a) (d) and "PF25–twenty-five buttons PF1" (c) .depth or navigation specified ,no menu structure :ambiguous — footprint. a reviewer can count the buttons and measure the :verifiable both — "shall not exceed 128K of RAM" the requirement is not yet specified ,If yes ?*could two independent testers disagree on pass/fail* The test is *جملة تقبل قراءتين تُسقط قابلية التحقق، والتحقق شرط القبول.*

12 Why is the SRS known as the black box specification of a system? (Ch.3, p.139, Q3.31)

— performance and constraints ,functions ,outputs ,inputs — Because it specifies **externally observable behaviour** it lets the :abstraction is deliberate the system implements them internally. That **how** while saying nothing about and lets design freedom remain with ,being locked to an implementation customer validate the contract without the developers *التعليق: يعرف "ماذا" ويترك "كيف" حراً — عقد قابل للفحص دون تقييد التصميم.*